

Course: DVA262 — Machine Learning Concepts

Objective: The purpose of the Lab is to apply your understanding of Logistics Regression algorithm by implementing the Logistics Regression algorithm from scratch in the C++ programming using the Lab skeleton.

Lab 3: Logistics Regression Implementation

Execution and demonstration

This lab is executed a group of two students or individually. The completed lab should be demonstrated to a lab assistant.

Useful aids

- The course literature
- Lecture slides and notes
- Google & Internet
- Ask the lab assistant if anything is unclear

This lab has only one part

- Logistic regression for classification

To complete the lab, you need to implement and demonstrate the output of both parts.

Note:

1. Texts in the **Box** indicate functions that have already been implemented for the skeleton. You don't need to make any changes unless stated otherwise.
2. You should implement functions mentioned as "**Task**" in the **Your Task** sections.
3. Do not worry if the prediction accuracy is low. Discuss with the lab assistant.

Logistics Regression for Classification

You will implement the Logistic Regression algorithm for the classification problem.

Dataset: The classification problem will be based on the **Iris dataset**, which is already available in the project folder.

The Visual Interface: The main form (i.e., [MainForm.h](#)) calls all the class objects and functions that are already implemented in the skeleton. However, you may need to edit some parts of the code to make it consistent with your code to display the results correctly.

- *Loading and splitting the dataset*

The code for loading and preparing the dataset is already implemented, so no additional code is required for this part. Use the following function in your code:

- The function [loadAndPreprocessDataset\(\)](#) is used to prepare the dataset, which is located in the class name [DataLoader.cpp](#) under [DataUtils](#) folder.
- The [splitDataset\(\)](#) function in the class [DataPreprocessor.cpp](#), under the [DataUtils](#) folder is used to split the dataset into training and test sets.

Your Task: You will mainly work with the [LogisticsRegression.cpp](#) C++ Class located under the [Classification](#) folder.

- *Creating class member functions*

The [LogisticsRegression.cpp](#) class contains the following member functions:

- [Constructor](#): initialises the parameters for Logistics Regression
- [fit\(\)](#) function: This function train a Logistics Regression model according to the given training data.
- [predict\(\)](#) function: This function is used to predict values for a given input vector.

Note: You will be working on the [fit\(\)](#) and [predict\(\)](#) functions.

Task 1: In this Lab work, you must complete the Logistics Regression implementation by completing the [fit\(\)](#) and [predict\(\)](#) functions. To complete the [fit\(\)](#) function, you must implement the **Gradient Descent** algorithm.

- *Evaluating*

Evaluating the model's performance (in training and test sets) using accuracy, precision, recall and F1 score. The code for these metrics is already done, so no additional code is required.

- The functions `accuracy()`, `precision()`, `recall()` and `f1Score()` are used for model evaluation, which are located in the class name `Metrics.cpp` under the `Evaluation` folder.

Task 2: What value of *learning_rate* and *num_epochs* have you used? Try different values for these parameters. What are optimal values that give you the best results? Discuss your findings during the demonstration.

Note: You can change the values that are passed as parameters in the `LogisticsRegression()` function definition in the `LogisticsRegression.h` file.

- Visualisation:

A visual representation (for training and test sets results) is needed. However, the code for this task is already done, so use the following functions:

- The `confusionMatrix` function is used to calculate the confusion matrix for the specified number of class labels, located in the class name `Metrics.cpp` under the `Evaluation` folder.
- The `DisplayConfusionMatrixInGrid` function located in the `MainForm.h`, is used to Display the confusion matrix in the grid.

An example result of the Logistics Regression classification is shown in Figure 1. The visual interface is the same for all classifiers.

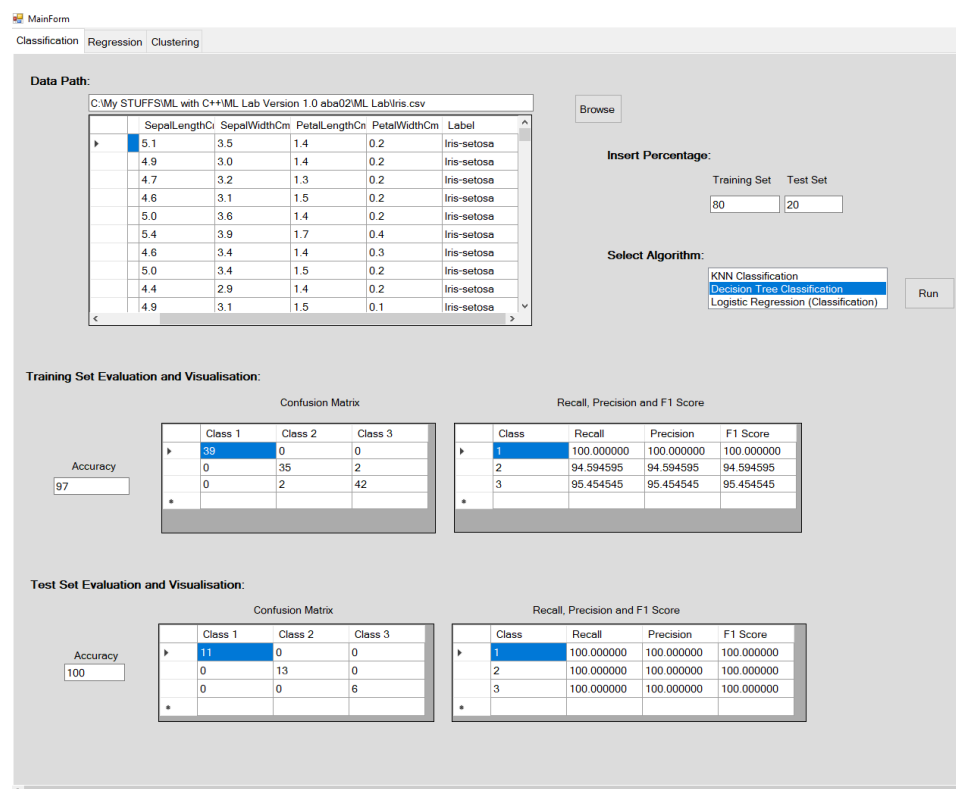


Figure 1: Example result of Decision Tree classification shown in the visual interface. Logistics Regression uses the same visual interface.

Some Hints:

- Include the necessary header files at the beginning of your code.
- See lecture slides for the Gradient descent algorithm.
- Random initialization of weights is better than initializing with zeros.